

Lab: Blind SQL injection with time delays

Platform: PortSwigger

Category: SQL Injection

Difficulty: Practitioner

Tools: Burp Suite

1. Objective

The goal of this lab is to exploit a blind SQL injection vulnerability in the TrackingId cookie and cause a 10-second delay in the application's response.

2. Impact

An attacker may be able to exploit the SQL injection to interact with the database even though the application does not directly display the query results. By using time delays, the attacker can infer information from the database based on how long the application takes to respond.

3. Steps to Solve

Step 1: Find the injection point

I opened the lab and browsed the website normally to generate some requests.

Then I opened **Burp Suite** → **Proxy** → **HTTP history** and looked for a request containing a TrackingId cookie.

I sent the request to **Repeater** so I could modify the cookie and test it.

Step 2: Test the SQL injection

In Repeater, I modified the TrackingId cookie and used a SQL payload that should cause the database to wait for 10 seconds.

- `TrackingId=U0CSRdcQs77JgM6R' || pg_sleep(10) --`

I sent the request and waited for the response.

The response took approximately **10 seconds** to return.

This confirmed that the SQL query was being executed and that I could control its execution time through the TrackingId cookie.

Step 3: Confirm the time delay

I sent the same request again to make sure the delay was not caused by a temporary network issue.

The application again took around **10 seconds** to respond.

This confirmed the blind SQL injection and completed the requirement of the lab.

4. Payload

The payload I used was: `TrackingId=U0CSRdcQs77JgM6R' || pg_sleep(10) --`

5. Result

The application took approximately **10 seconds** to respond after sending the payload. This confirmed that the SQL injection was successfully exploited using a time delay

6. Root Cause

The vulnerability exists because the application uses the TrackingId cookie value directly in a SQL query without properly parameterizing the query.

Although the application does not return the SQL results or show different responses based on the query result, the database still executes the injected SQL.

7. Conclusions and Evaluation

What did I learn?

I learned how blind SQL injection can be detected using time delays when the application does not return database results directly. I also learned how `pg_sleep()` can be used to confirm that an injected SQL query is being executed.

Lab Rating

4/5 ★★★★★

Did I face any challenges?

The main challenge was understanding how to confirm a blind SQL injection when there is no visible database output or error message. The response time was the main indicator that the injected SQL was being executed.

8. References

- PortSwigger Web Security Academy - SQL Injection
 - PortSwigger Web Security Academy - Blind SQL Injection
 - PortSwigger Web Security Academy - Blind SQL injection with time delays
 - OWASP SQL Injection Prevention Cheat Sheet
-

9. Screenshots

The first screenshot shows the Burp Suite interface with the 'HTTP history' tab selected. The history table lists two requests:

#	Host	Method	URL	Params	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Time	Listener port	Start respons...
5978	https://0af900cb039ae77f80e...	GET	/academyLabHeader		101	147					✓	79.125.84.16	02:18:27 28 ...	8080	91
5977	https://0af900cb039ae77f80e...	GET	/		200	11609	HTML		Blind SQL injection with ...		✓	79.125.84.16	02:18:26 28 ...	8080	124

The 'Request' tab shows the raw HTTP request for the second entry. The 'Response' tab shows the HTML response, which includes the 'WebSecurity Academy' logo and a 'Blind SQL injection with time delays' lab card. The lab card indicates 'LAB Not solved'.

The second screenshot shows the same interface after the lab has been solved. The 'Blind SQL injection with time delays' lab card now indicates 'LAB Solved'. The 'Request' tab shows the raw HTTP request, and the 'Response' tab shows the HTML response, which includes a 'Congratulations, you solved the lab!' message.